

Model Development and Testing

```
In [21]: import torch
import torch.optim as optim
import torch.nn as nn
import pandas as pd
import numpy as np
from collections import OrderedDict
```

Data preparation

The first step in the process is to format the data for a neural network based model. All of the categorical variables must be one-hot encoded.

Step 1: Determine categorical and quantitative variables. This is easy to do based on the output of the describe function from pandas.

```
In [22]: data = pd.read_csv('project_data.csv')

quantitative = set((data.drop('Booking_ID', axis=1).describe().columns))
categorical = set((data.drop(['Booking_ID', 'arrival_date', 'booking_status'], axis=1).columns))
```

Step 2: Convert the categorical variables to one-hot encoded versions. Pandas has a built in "get dummies" function to do this, but I am feeling ambitious. Let's implement this from scratch.

```
In [23]: for cat in list(categorical):
    # Find the unique values for the column
    unique_vals = data[cat].unique()

    # For each unique value, create a new column with the value name
    for val in unique_vals:
        data[val] = np.where(data[cat].values==val, 1, 0)
```

```
In [24]: encoded_data = data.drop(list(categorical)+['Booking_ID'], axis = 1, inplace=False)
```

```
In [25]: print(encoded_data.head(5))
```

	no_of_adults	no_of_children	no_of_weekend_nights	no_of_week_nights	\
0	2	0	1	2	
1	2	0	2	3	
2	1	0	2	1	
3	2	0	0	2	
4	2	0	1	1	

	required_car_parking_space	lead_time	arrival_date	repeated_guest	\
0	0	224	2017-10-02	0	
1	0	5	2018-11-06	0	
2	0	1	2018-02-28	0	
3	0	211	2018-05-20	0	
4	0	48	2018-04-11	0	

	no_of_previous_cancellations	no_of_previous_bookings_not_canceled	...	\
0	0		0	...
1	0		0	...
2	0		0	...
3	0		0	...
4	0		0	...

	room_type4	room_type2	room_type6	room_type5	room_type7	room_type3	\
0	0	0	0	0	0	0	
1	0	0	0	0	0	0	
2	0	0	0	0	0	0	
3	0	0	0	0	0	0	
4	0	0	0	0	0	0	

	meal_plan_1	not_selected	meal_plan_2	meal_plan_3
0	1	0	0	0
1	0	1	0	0
2	1	0	0	0
3	1	0	0	0
4	0	1	0	0

[5 rows x 29 columns]

Now that the categorical variables have been one-hot encoded, we can tackle the arrival_date variable. I want to extract a the month from this because I suspect there may be seasonality to the data within this time frame.

```
In [26]: encoded_data['arrival_date'] = [int(date.split('-')[1]) for date in encoded_data['arrival_date']]
```

```
In [27]: encoded_data['arrival_date']
```

```
Out[27]: 0      10
1      11
2       2
3       5
4       4
..
36233   8
36234  10
36235   7
36236   4
36237  12
Name: arrival_date, Length: 36238, dtype: int64
```

Now the booking status variable must be transformed to a numerical representation.

```
In [28]: encoded_data[encoded_data['booking_status'] == "not_canceled"] = 0
encoded_data[encoded_data['booking_status'] == "canceled"] = 1
```

Looks like everything is model ready. Let's take a look at the dataset again to be sure.

```
In [29]: encoded_data.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 36238 entries, 0 to 36237
Data columns (total 29 columns):
 #   Column                                Non-Null Count  Dtype
---  -
 0   no_of_adults                          36238 non-null  int64
 1   no_of_children                        36238 non-null  int64
 2   no_of_weekend_nights                  36238 non-null  int64
 3   no_of_week_nights                     36238 non-null  int64
 4   required_car_parking_space            36238 non-null  int64
 5   lead_time                             36238 non-null  int64
 6   arrival_date                          36238 non-null  int64
 7   repeated_guest                        36238 non-null  int64
 8   no_of_previous_cancellations          36238 non-null  int64
 9   no_of_previous_bookings_not_canceled  36238 non-null  int64
10   avg_price_per_room                    36238 non-null  float64
11   no_of_special_requests                36238 non-null  int64
12   booking_status                        36238 non-null  object
13   offline                               36238 non-null  int64
14   online                                36238 non-null  int64
15   corporate                             36238 non-null  int64
16   aviation                              36238 non-null  int64
17   complementary                         36238 non-null  int64
18   room_type1                            36238 non-null  int64
19   room_type4                            36238 non-null  int64
20   room_type2                            36238 non-null  int64
21   room_type6                            36238 non-null  int64
22   room_type5                            36238 non-null  int64
23   room_type7                            36238 non-null  int64
24   room_type3                            36238 non-null  int64
25   meal_plan_1                           36238 non-null  int64
26   not_selected                          36238 non-null  int64
27   meal_plan_2                           36238 non-null  int64
28   meal_plan_3                           36238 non-null  int64
dtypes: float64(1), int64(27), object(1)
memory usage: 8.0+ MB
```

Beautiful...

Model Development

I want to build a model handling framework to make it easier to test and compare different model architectures. To do so, I will build a Model Handler class.

```
In [30]: import matplotlib.pyplot as plt
from time import time
from sklearn.metrics import ConfusionMatrixDisplay, accuracy_score, roc_curve, roc_auc_score, pr...
```

```

In [ ]: class ModelHandler():
    def __init__(self, X: pd.DataFrame, Y: pd.Series, architecture: nn.Sequential, optimizer: op
        self.X = self._transform_data(X)
        self.Y = self._transform_data(Y, torch.long)
        self._split_data()
        self.model = architecture
        self.optimizer = optimizer
        self.loss_fn = loss_fn().to('cuda')

        # Utilize the gpu for training
        self.model.to('cuda')
        self.loss_fn.to('cuda')

    def train(self, n_epochs, lr = 1e-3, slope_interval = 10, stop_slope = 0.000001):
        self.training_losses = []
        self.test_losses = []
        self.optimizer = self.optimizer(self.model.parameters(), lr)
        start_time = time()

        for epoch in range(1, n_epochs+1):
            p_train = self.model(self.X_train).to('cuda')
            train_loss = self.loss_fn(p_train, self.Y_train)
            self.training_losses.append(train_loss.to('cpu').item())

            p_test = self.model(self.X_test)
            loss_test = self.loss_fn(p_test, self.Y_test)
            self.test_losses.append(loss_test.to('cpu').item())

            self.optimizer.zero_grad()
            train_loss.backward() # <2>
            self.optimizer.step()

            if len(self.training_losses)>slope_interval:
                self.train_loss_slope = ((self.training_losses[-1]-self.training_losses[-slope_in
                self.test_loss_slope = ((self.training_losses[-1]-self.training_losses[-slope_in
                print(f"Epoch {epoch}\tTraining loss {train_loss.item():.4f}\tTest loss {loss_
                    Train Slope: {self.train_loss_slope:.20f}\tTest Slope: {self.test_loss_slo

                if abs(self.train_loss_slope) < stop_slope and abs(self.test_loss_slope) < stop_
                    break

            end_time = time()
            print('\n')
            print(f"Total training time: {end_time - start_time} seconds")
            print('\n')
            self._plot_losses(self.training_losses, self.test_losses, 'Training vs. Test Loss')

    def eval_model(self, labels):
        self.labels = labels

        self.confusion_matrix = {
            'true_positive':0,
            'false_positive':0,
            'true_negative':0,
            'false_negative':0
        }

        self.predictions = []
        correct_preds = 0

```

```

start_time = time()
for idx in range(len(self.X_val)):
    probs = torch.exp(self.model(self.X_val[idx]))
    pred = torch.argmax(probs).to('cpu').item()
    self.predictions.append(pred)
    true_val = self.Y_val[idx].item()
    #print(pred, true_val)
    if pred == true_val:
        correct_preds+=1
end_time = time()
print(f"Inference time for {len(self.X_val)} predictions: {end_time-start_time} seconds")
print('\n')

for i, dir in enumerate(self.Y_val):
    if self.predictions[i] == 1 and dir == 0:
        self.confusion_matrix['false_positive']+=1
    elif self.predictions[i] == 1 and dir == 1:
        self.confusion_matrix['true_positive']+=1
    elif self.predictions[i] == 0 and dir == 1:
        self.confusion_matrix['false_negative']+=1
    elif self.predictions[i] == 0 and dir == 0:
        self.confusion_matrix['true_negative'] += 1

self.cm_array = np.array([[self.confusion_matrix['true_negative'], self.confusion_matrix[
    self.confusion_matrix['false_negative'], self.confusion_matr

self._describe_predictions()
self._compute_metrics()
self._plot_cm()

def _plot_losses(self, train_losses, test_losses, title):
    x = np.array([i for i in range(1, len(train_losses)+1)])
    y1 = np.array(train_losses)
    y2 = np.array(test_losses)

    plt.plot(x, y1, label = "Train Loss")
    plt.plot(x, y2, label = "Test Loss")
    plt.legend(loc="upper right")
    plt.title(title)

    plt.show()

def _split_data(self):
    n_samples = len(self.X)
    n_test = int(0.20 * n_samples)
    n_val = int(0.10 * n_samples)

    shuffled_indices = torch.randperm(n_samples)

    train_indices = shuffled_indices[:-n_val-n_test]
    test_indices = shuffled_indices[-n_val-n_test:-n_val]
    val_indices = shuffled_indices[-n_val:]

    # Verify the indices are all unique
    train_set = set(train_indices.numpy())
    test_set = set(test_indices.numpy())
    val_set = set(val_indices.numpy())

```

```

if not len(train_set & test_set) and not len(train_set & val_set) and not len(test_set &
    print("Train, Test, and Validation data is verified as unique.")
else:
    print("DATA LEAK.")

#Assign train data
self.X_train = self.X[train_indices].to('cuda')
self.Y_train = self.Y[train_indices].to('cuda')

#Assign train data
self.X_test = self.X[test_indices].to('cuda')
self.Y_test = self.Y[test_indices].to('cuda')

#Assign val data
self.X_val = self.X[val_indices].to('cuda')
self.Y_val = self.Y[val_indices].to('cuda')

def _transform_data(self, data, type = torch.float):
    return torch.tensor(data.to_numpy()).to(type)

def _describe_predictions(self):
    print(f'Number of {self.labels[1]} Predictions: {self.confusion_matrix[f'true_positive']}')
    print(f'Number of {self.labels[0]} Predictions: {self.confusion_matrix['true_negative']}')
    print(f'Percent {self.labels[1]} Predictions: {((self.confusion_matrix['true_positive']
    print(f'Percent {self.labels[0]} Predictions: {((self.confusion_matrix['true_negative']

def _plot_cm(self):
    fig, axes = plt.subplots(1,2,figsize=(15,10))
    axes = axes.flatten()

    cm = ConfusionMatrixDisplay(
        confusion_matrix=self.cm_array,
        display_labels=self.labels
    )

    cm.plot(cmap=plt.cm.Blues,ax=axes[0])

    axes[0].set_title('Confusion Matrix')
    axes[0].set_xlabel('Predicted Label')
    axes[0].set_ylabel('True Label')

    axes[1].plot(self.fpr, self.tpr, color='darkorange', lw=2, label=f'ROC curve (AUC = {self
    axes[1].plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--', label='Random Chance')
    axes[1].set_xlim([0.0, 1.0])
    axes[1].set_ylim([0.0, 1.05])
    axes[1].set_xlabel('False Positive Rate')
    axes[1].set_ylabel('True Positive Rate')
    axes[1].set_title('Receiver Operating Characteristic (ROC) Curve')
    axes[1].legend(loc="lower right")
    axes[1].grid(True)

    plt.show()

def _compute_metrics(self):
    Y = self.Y_val.cpu()

    # Calculate the ROC curve points
    self.fpr, self.tpr, self.thresholds = roc_curve(Y, self.predictions)

```

```

# Calculate the AUC
self.auc = roc_auc_score(Y, self.predictions)

self.accuracy = f'{accuracy_score(Y, self.predictions)*100:.2f}%'

# Compute recall
self.recall = self.confusion_matrix['true_positive'] / (self.confusion_matrix['true_posi

# Compute precision
self.precision = precision_score(Y, self.predictions)

# Compute F1-Score
self.f1_score = f'{(2*self.precision*self.recall)/(self.precision+self.recall):.2f}'

print(f'\nAccuracy: {self.accuracy}')
print(f'\nRecall: {self.recall}')
print(f'\nPrecision: {self.precision}')
print(f'\nF1-Score: {self.f1_score}')

```

Set up some variables that will be used across all models.

```

In [32]: torch.seed()
X = encoded_data.drop(["booking_status"], axis=1)
Y = encoded_data['booking_status'].astype(np.int64)
input_features = len(encoded_data.drop('booking_status', axis=1).columns)
labels = ['Not Canceled', 'Canceled']

```

Start with a very basic model that uses MSELoss and a linear output layer.

```

In [43]: model_1 = nn.Sequential(OrderedDict([
    ('hidden_linear1', nn.Linear(in_features=input_features, out_features=10)),
    ('dropout1', nn.Dropout(0.2)),
    ('hidden_linear2', nn.Linear(in_features=10, out_features=5)),
    ('hidden_activation', nn.Tanh()),
    ('output_linear', nn.Linear(in_features=5, out_features=2))
]))

model_1 = ModelHandler(
    X = X,
    Y = Y,
    architecture = model_1,
    optimizer = optim.SGD,
    loss_fn = nn.CrossEntropyLoss
)

model_1.train(100000, 1e-4, 100, 1e-4)
model_1.eval_model(labels)

```

Train, Test, and Validation data is verified as unique.

Epoch 101 Training loss 0.6357 Test loss 0.6351 Train Slope: -0.00002197802066802
979 Test Slope: -0.00002197802066802979

Total training time: 0.08760285377502441 seconds



Inference time for 3623 predictions: 0.7168064117431641 seconds

Number of Canceled Predictions: 1079

Number of Not Canceled Predictions: 2544

Percent Canceled Predictions: 29.781948661330386

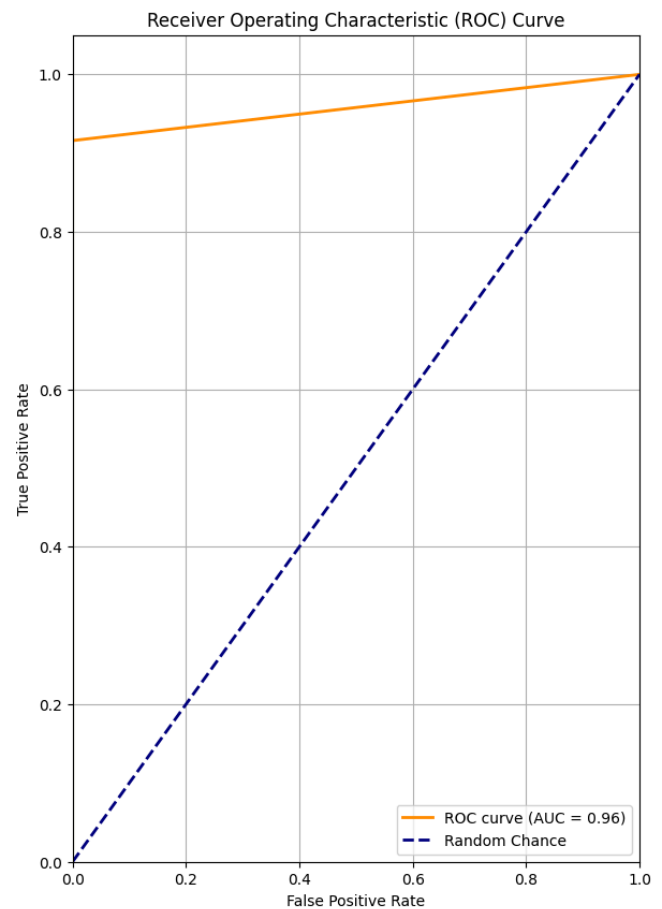
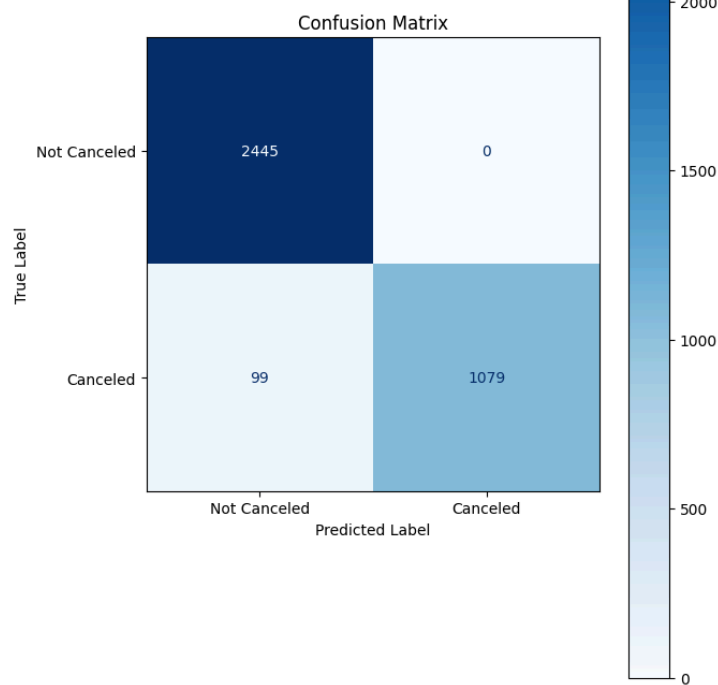
Percent Not Canceled Predictions: 70.21805133866961

Accuracy: 97.27%

Recall: 0.9159592529711376

Precision: 1.0

F1-Score: 0.96



```
In [34]: model_2 = nn.Sequential(OrderedDict([
    ('hidden_linear1', nn.Linear(in_features=input_features, out_features=200)),
    ('hidden_linear2', nn.Linear(in_features=200, out_features=500)),
    ('relu1', nn.ReLU()),
    ('dropout1', nn.Dropout(0.2)),
    ('hidden_linear3', nn.Linear(in_features=500, out_features=400)),
    ('relu2', nn.ReLU()),
    ('dropout2', nn.Dropout(0.3)),
    ('hidden_linear4', nn.Linear(in_features=400, out_features=200)),
    ('lrelu1', nn.LeakyReLU()),
    ('dropout3', nn.Dropout(0.3)),
    ('hidden_linear6', nn.Linear(in_features=200, out_features=150)),
    ('relu3', nn.ReLU()),
    ('dropout4', nn.Dropout(0.2)),
    ('hidden_linear7', nn.Linear(in_features=150, out_features=100)),
    ('hidden_linear8', nn.Linear(in_features=100, out_features=50)),
    ('relu4', nn.ReLU()),
    ('dropout5', nn.Dropout(0.3)),
    ('hidden_linear9', nn.Linear(in_features=50, out_features=20)),
    ('lrelu2', nn.LeakyReLU()),
    ('dropout6', nn.Dropout(0.3)),
    ('hidden_linear10', nn.Linear(in_features=20, out_features=2)),
    ('output', nn.Softmax())
]))
```

```
model_2 = ModelHandler(
    X = X,
    Y = Y,
    architecture = model_2,
    optimizer = optim.AdamW,
    loss_fn = nn.CrossEntropyLoss
```

```
)  
  
model_2.train(100000, 1e-4, 100, 1e-4)  
model_2.eval_model(labels)
```

Train, Test, and Validation data is verified as unique.

c:\Users\water\anaconda3\envs\DSE6211\Lib\site-packages\torch\nn\modules\module.py:1773: UserWarning: Implicit dimension choice for softmax has been deprecated. Change the call to include dim=X as an argument.

```
    return self._call_impl(*args, **kwargs)
```

Epoch 201 Training loss 0.3140 Test loss 0.3141 Train Slope: -0.00009971737861633
301 Test Slope: -0.00009971737861633301

Total training time: 1.4564611911773682 seconds



Inference time for 3623 predictions: 1.850196361541748 seconds

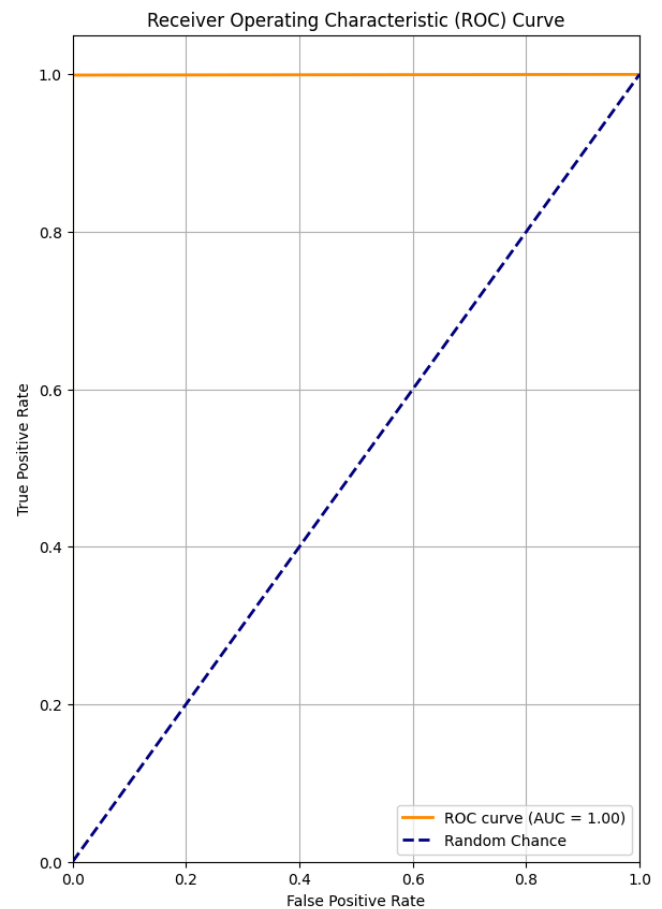
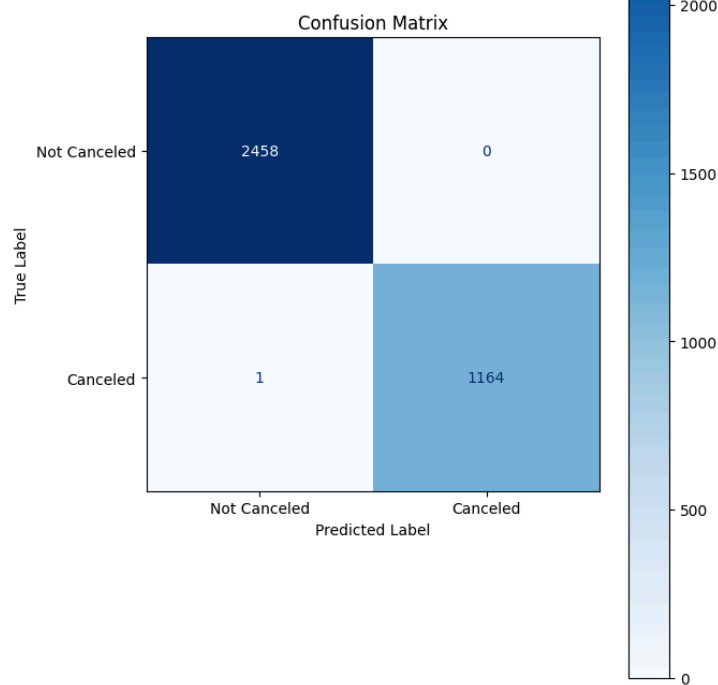
Number of Canceled Predictions: 1164
Number of Not Canceled Predictions: 2459
Percent Canceled Predictions: 32.1280706596743
Percent Not Canceled Predictions: 67.87192934032569

Accuracy: 99.97%

Recall: 0.9991416309012876

Precision: 1.0

F1-Score: 1.00



```
In [35]: model_3 = nn.Sequential(OrderedDict([
    ('hidden_linear1', nn.Linear(in_features=input_features, out_features=20)),
    ('hidden_linear2', nn.Linear(in_features=20, out_features=50)),
    ('dropout1', nn.Dropout(0.2)),
    ('hidden_linear3', nn.Linear(in_features=50, out_features=40)),
    ('dropout2', nn.Dropout(0.3)),
    ('hidden_linear4', nn.Linear(in_features=40, out_features=20)),
    ('hidden_activation', nn.LeakyReLU()),
    ('hidden_linear6', nn.Linear(in_features=20, out_features=2)),
    ('output', nn.LogSoftmax())
]))

model_3 = ModelHandler(
    X = X,
    Y = Y,
    architecture = model_3,
    optimizer = optim.AdamW,
    loss_fn = nn.NLLLoss
)

model_3.train(100000, 1e-4, 100, 1e-4)
model_3.eval_model(labels)
```

Train, Test, and Validation data is verified as unique.

Epoch 149 Training loss 0.3821 Test loss 0.3775

Train Slope: -0.00206611007452011

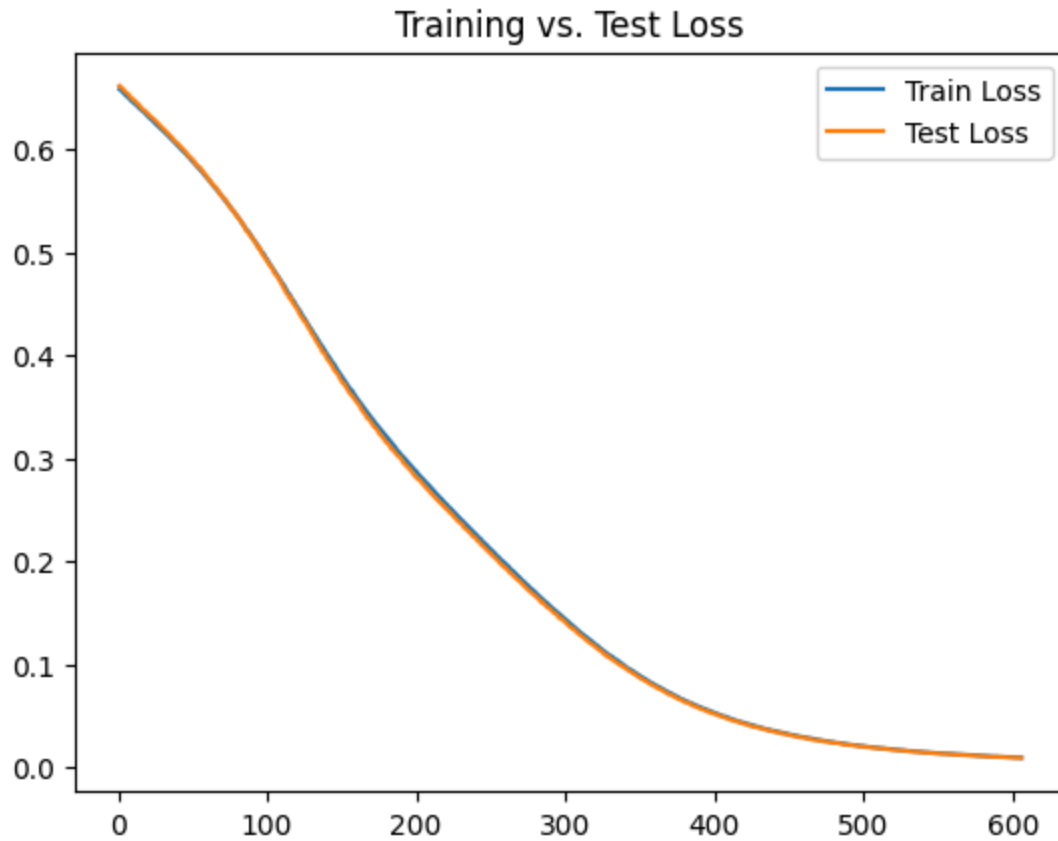
091 Test Slope: -0.00206611007452011091

c:\Users\water\anaconda3\envs\DSE6211\Lib\site-packages\torch\nn\modules\module.py:1773: UserWarning: Implicit dimension choice for log_softmax has been deprecated. Change the call to include dim=X as an argument.

```
return self._call_impl(*args, **kwargs)
```

Epoch 606 Training loss 0.0095 Test loss 0.0094 Train Slope: -0.00009957513771951
199 Test Slope: -0.00009957513771951199

Total training time: 0.8250422477722168 seconds



Inference time for 3623 predictions: 0.9488115310668945 seconds

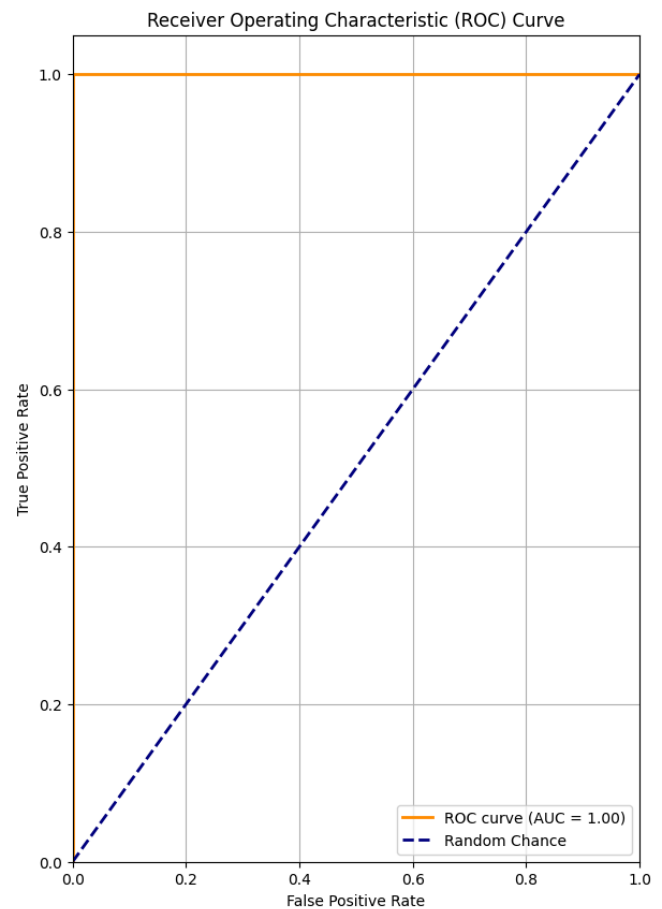
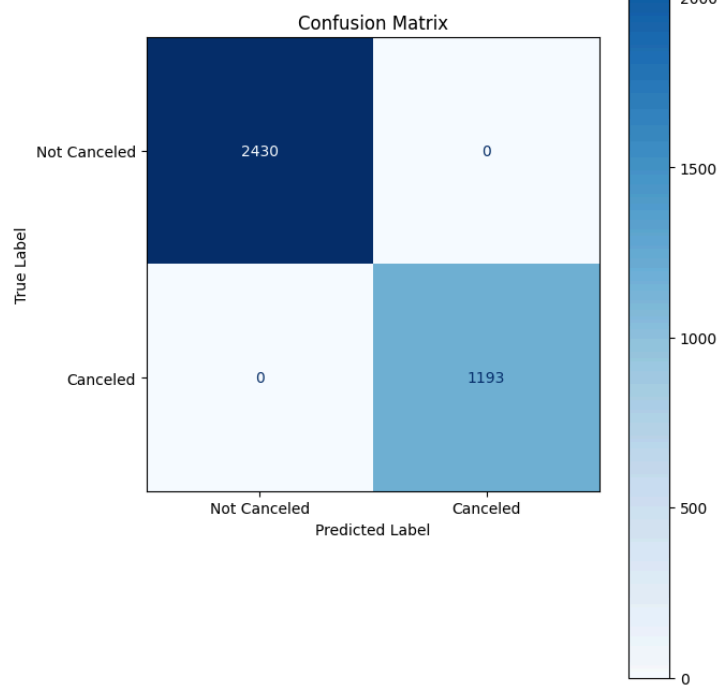
Number of Canceled Predictions: 1193
Number of Not Canceled Predictions: 2430
Percent Canceled Predictions: 32.92851228263869
Percent Not Canceled Predictions: 67.0714877173613

Accuracy: 100.00%

Recall: 1.0

Precision: 1.0

F1-Score: 1.00



```
In [36]: model_4 = nn.Sequential(OrderedDict([
    ('hidden_linear1', nn.Linear(in_features=input_features, out_features=10)),
    ('hidden_linear2', nn.Linear(in_features=10, out_features=20)),
    ('hidden_linear3', nn.Linear(in_features=20, out_features=30)),
    ('hidden_linear4', nn.Linear(in_features=30, out_features=20)),
    ('hidden_linear5', nn.Linear(in_features=20, out_features=2)),
    ('output', nn.LogSoftmax())
]))

model_4 = ModelHandler(
    X = X,
    Y = Y,
    architecture = model_4,
    optimizer = optim.AdamW,
    loss_fn = nn.NLLLoss
)

model_4.train(100000, 1e-4, 100, 1e-4)
model_4.eval_model(labels)
```

Train, Test, and Validation data is verified as unique.

Epoch 198 Training loss 0.3207 Test loss 0.3207

Train Slope: -0.00254373848438262

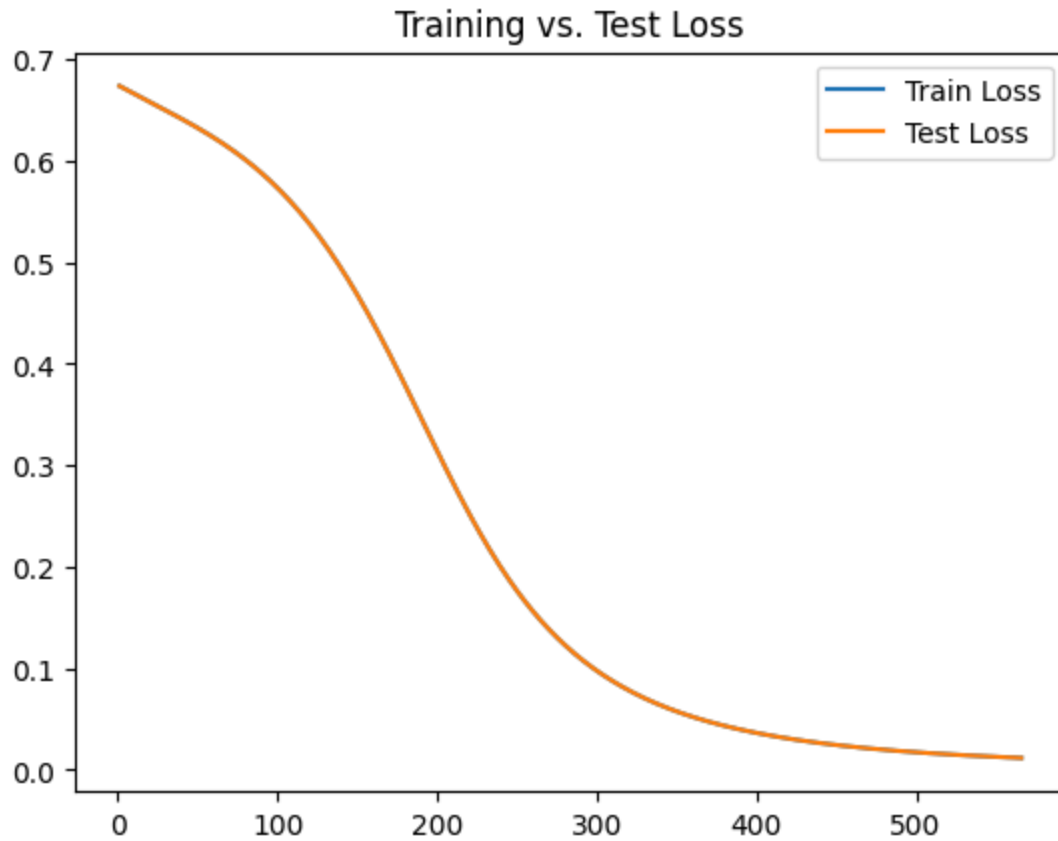
931 Test Slope: -0.00254373848438262931

c:\Users\water\anaconda3\envs\DSE6211\Lib\site-packages\torch\nn\modules\module.py:1773: UserWarning: Implicit dimension choice for log_softmax has been deprecated. Change the call to include dim=X as an argument.

return self._call_impl(*args, **kwargs)

Epoch 566 Training loss 0.0116 Test loss 0.0116 Train Slope: -0.00009928536601364
613 Test Slope: -0.00009928536601364613

Total training time: 0.6358087062835693 seconds



Inference time for 3623 predictions: 0.8339002132415771 seconds

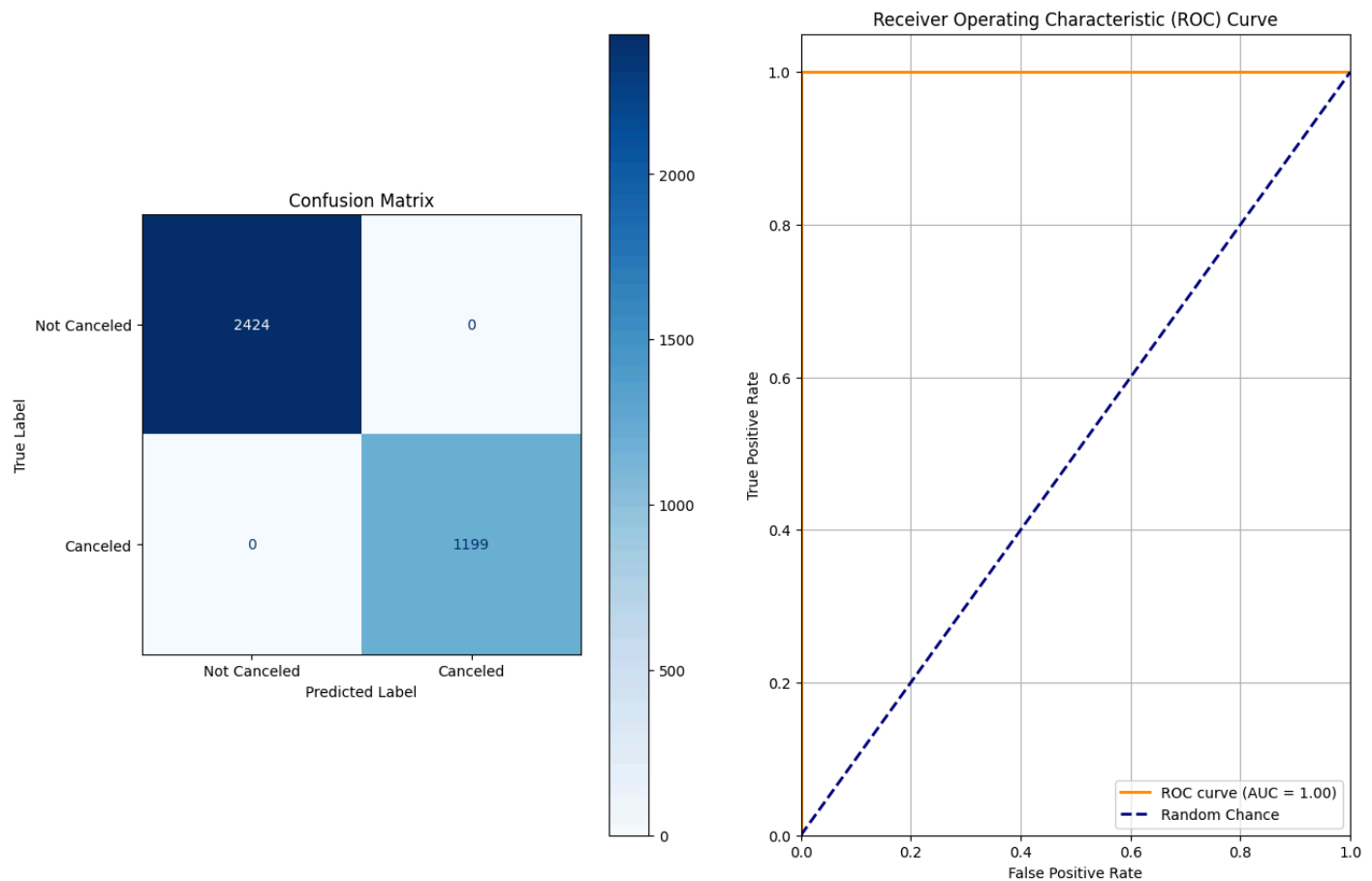
Number of Canceled Predictions: 1199
Number of Not Canceled Predictions: 2424
Percent Canceled Predictions: 33.094120894286505
Percent Not Canceled Predictions: 66.9058791057135

Accuracy: 100.00%

Recall: 1.0

Precision: 1.0

F1-Score: 1.00



Trying to overfit the models

```
In [37]: X = encoded_data.drop(["booking_status"], axis=1)
Y = encoded_data['booking_status'].astype(np.int64)
input_features = len(encoded_data.drop('booking_status', axis=1).columns)
labels = ['Not Canceled', 'Canceled']
```

```
In [38]: model_1 = nn.Sequential(OrderedDict([
    ('hidden_linear1', nn.Linear(in_features=input_features, out_features=10)),
    ('dropout1', nn.Dropout(0.2)),
    ('hidden_linear2', nn.Linear(in_features=10, out_features=5)),
    ('hidden_activation', nn.Tanh()),
    ('output_linear', nn.Linear(in_features=5, out_features=2))
]))

model_1 = ModelHandler(
    X = X,
    Y = Y,
    architecture = model_1,
    optimizer = optim.SGD,
    loss_fn = nn.CrossEntropyLoss
)

model_1.train(100000000000, 1e-4, 100, 1e-10)
model_1.eval_model(labels)
```

Train, Test, and Validation data is verified as unique.

Epoch 25057 Training loss 0.1179 Test loss 0.1170
597 Test Slope: -0.0000000007450580597

Train Slope: -0.0000000007450580

Total training time: 21.7206449508667 seconds



Inference time for 3623 predictions: 0.6423470973968506 seconds

Number of Canceled Predictions: 1209

Number of Not Canceled Predictions: 2414

Percent Canceled Predictions: 33.37013524703284

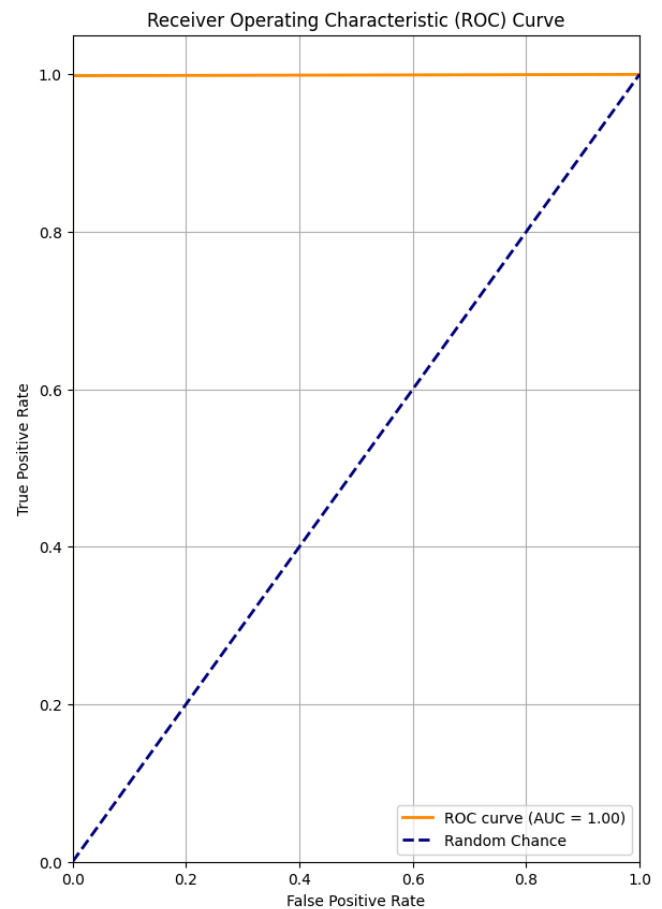
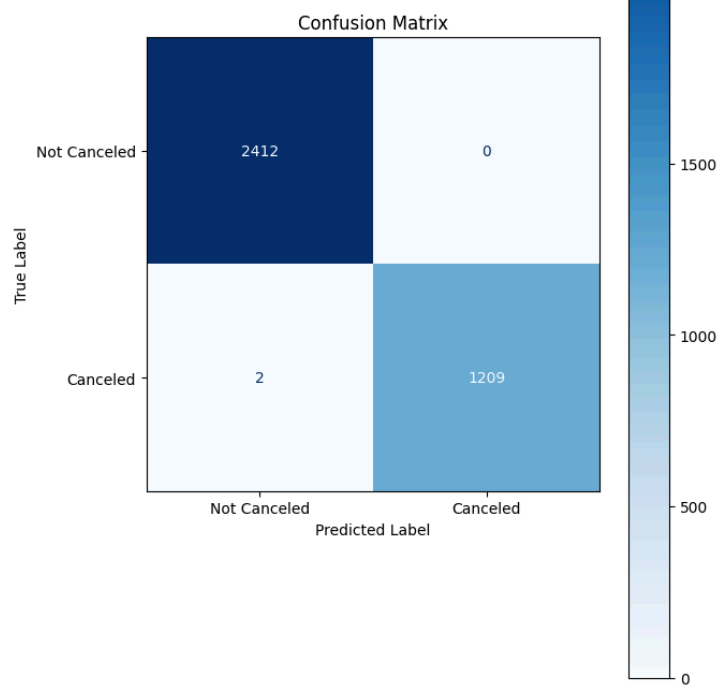
Percent Not Canceled Predictions: 66.62986475296715

Accuracy: 99.94%

Recall: 0.9983484723369116

Precision: 1.0

F1-Score: 1.00



```
In [39]: model_2 = nn.Sequential(OrderedDict([
    ('hidden_linear1', nn.Linear(in_features=input_features, out_features=200)),
    ('hidden_linear2', nn.Linear(in_features=200, out_features=500)),
    ('relu1', nn.ReLU()),
    ('dropout1', nn.Dropout(0.2)),
    ('hidden_linear3', nn.Linear(in_features=500, out_features=400)),
    ('relu2', nn.ReLU()),
    ('dropout2', nn.Dropout(0.3)),
    ('hidden_linear4', nn.Linear(in_features=400, out_features=200)),
    ('lrelu1', nn.LeakyReLU()),
    ('dropout3', nn.Dropout(0.3)),
    ('hidden_linear6', nn.Linear(in_features=200, out_features=150)),
    ('relu3', nn.ReLU()),
    ('dropout4', nn.Dropout(0.2)),
    ('hidden_linear7', nn.Linear(in_features=150, out_features=100)),
    ('hidden_linear8', nn.Linear(in_features=100, out_features=50)),
    ('relu4', nn.ReLU()),
    ('dropout5', nn.Dropout(0.3)),
    ('hidden_linear9', nn.Linear(in_features=50, out_features=20)),
    ('lrelu2', nn.LeakyReLU()),
    ('dropout6', nn.Dropout(0.3)),
    ('hidden_linear10', nn.Linear(in_features=20, out_features=2)),
    ('output', nn.Softmax())
]))
```

```
model_2 = ModelHandler(
    X = X,
    Y = Y,
    architecture = model_2,
    optimizer = optim.AdamW,
    loss_fn = nn.CrossEntropyLoss
```

```
)  
  
model_2.train(10000000000, 1e-4, 100, 0.00000000001)  
model_2.eval_model(labels)
```

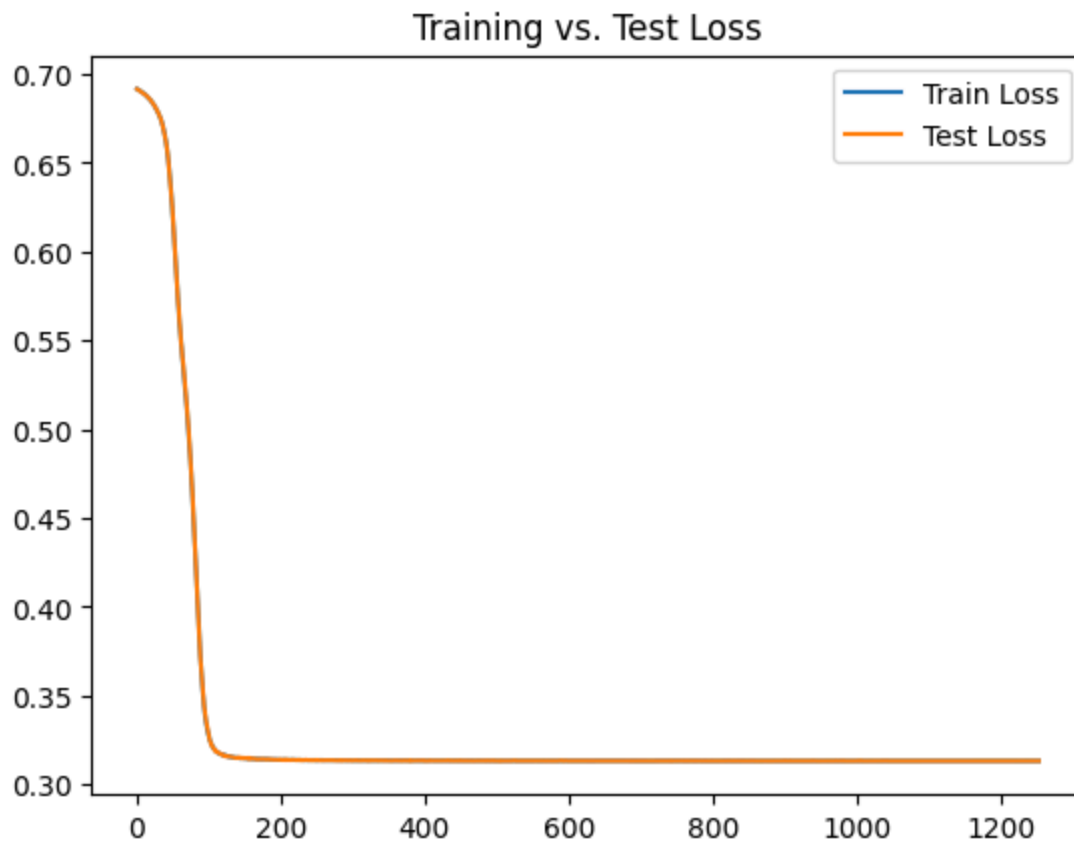
Train, Test, and Validation data is verified as unique.

c:\Users\water\anaconda3\envs\DSE6211\Lib\site-packages\torch\nn\modules\module.py:1773: UserWarning: Implicit dimension choice for softmax has been deprecated. Change the call to include dim=X as an argument.

```
    return self._call_impl(*args, **kwargs)
```

Epoch 1253 Training loss 0.3133 Test loss 0.3133 Train Slope: 0.0000000000000000
00 Test Slope: 0.00000000000000000064

Total training time: 8.576592445373535 seconds



Inference time for 3623 predictions: 1.7532508373260498 seconds

Number of Canceled Predictions: 1162

Number of Not Canceled Predictions: 2461

Percent Canceled Predictions: 32.072867789125034

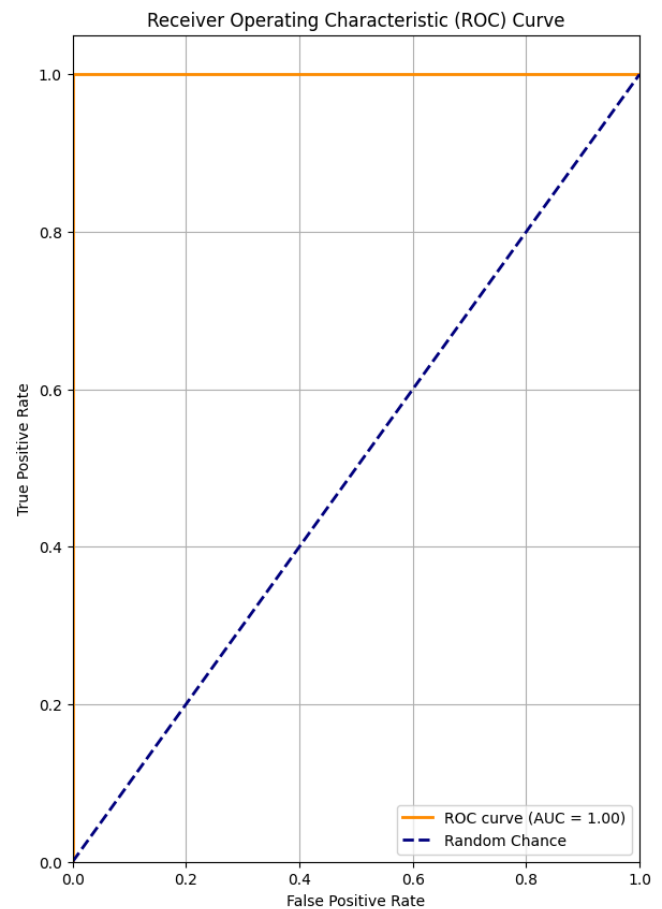
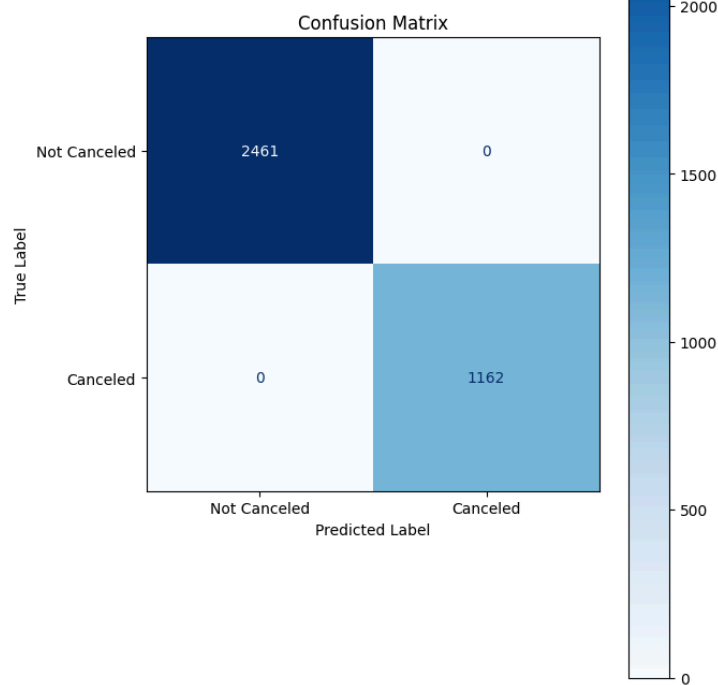
Percent Not Canceled Predictions: 67.92713221087496

Accuracy: 100.00%

Recall: 1.0

Precision: 1.0

F1-Score: 1.00



```
In [40]: model_3 = nn.Sequential(OrderedDict([
    ('hidden_linear1', nn.Linear(in_features=input_features, out_features=20)),
    ('hidden_linear2', nn.Linear(in_features=20, out_features=50)),
    ('dropout1', nn.Dropout(0.2)),
    ('hidden_linear3', nn.Linear(in_features=50, out_features=40)),
    ('dropout2', nn.Dropout(0.3)),
    ('hidden_linear4', nn.Linear(in_features=40, out_features=20)),
    ('hidden_activation', nn.LeakyReLU()),
    ('hidden_linear6', nn.Linear(in_features=20, out_features=2)),
    ('output', nn.LogSoftmax())
]))
```

```
model_3 = ModelHandler(
    X = X,
    Y = Y,
    architecture = model_3,
    optimizer = optim.AdamW,
    loss_fn = nn.NLLLoss
)
```

```
model_3.train(1000000, 1e-4, 100, 0.00000000000000000001)
model_3.eval_model(labels)
```

Train, Test, and Validation data is verified as unique.

Epoch 141 Training loss 0.4356 Test loss 0.4346

Train Slope: -0.00233659625053405

772 Test Slope: -0.00233659625053405772

c:\Users\water\anaconda3\envs\DSE6211\Lib\site-packages\torch\nn\modules\module.py:1773: UserWarning: Implicit dimension choice for log_softmax has been deprecated. Change the call to include dim=X as an argument.

```
return self._call_impl(*args, **kwargs)
```

Epoch 18796 Training loss 0.0000 Test loss 0.0000 Train Slope: 0.0000000000000000
00 Test Slope: 0.00000000000000000024

Total training time: 24.942312955856323 seconds



Inference time for 3623 predictions: 0.8966593742370605 seconds

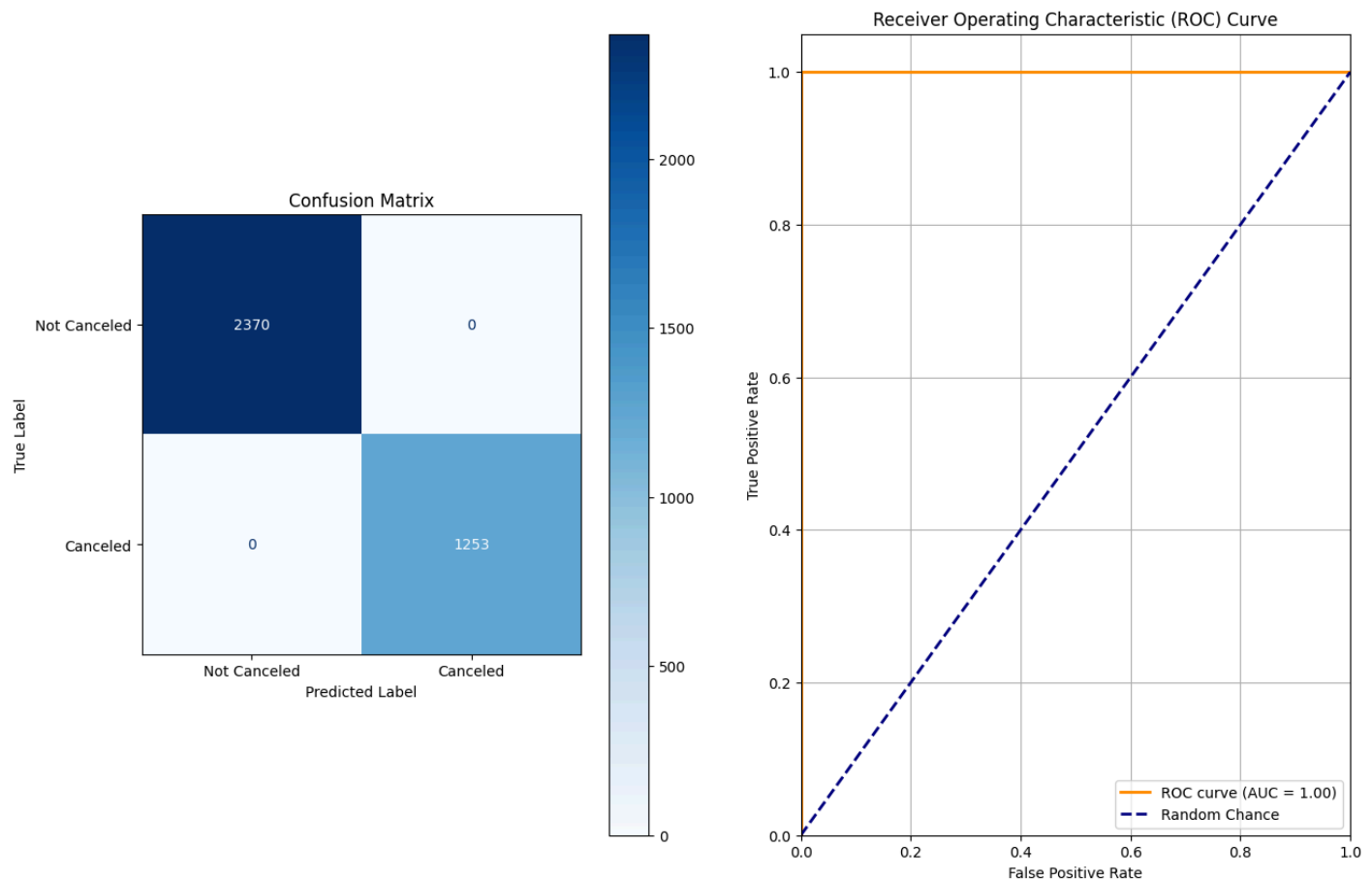
Number of Canceled Predictions: 1253
Number of Not Canceled Predictions: 2370
Percent Canceled Predictions: 34.58459839911676
Percent Not Canceled Predictions: 65.41540160088324

Accuracy: 100.00%

Recall: 1.0

Precision: 1.0

F1-Score: 1.00



New Model Architecture

```
In [41]: model_4 = nn.Sequential(OrderedDict([
    ('hidden_linear1', nn.Linear(in_features=input_features, out_features=10)),
    ('hidden_linear2', nn.Linear(in_features=10, out_features=20)),
    ('hidden_linear3', nn.Linear(in_features=20, out_features=30)),
    ('hidden_linear4', nn.Linear(in_features=30, out_features=20)),
    ('hidden_linear5', nn.Linear(in_features=20, out_features=2)),
    ('output', nn.LogSoftmax())
]))

model_4 = ModelHandler(
    X = X,
    Y = Y,
    architecture = model_4,
    optimizer = optim.AdamW,
    loss_fn = nn.NLLLoss
)

model_4.train(1000000, 1e-4, 1000, 1e-20)
model_4.eval_model(labels)
```

Train, Test, and Validation data is verified as unique.

c:\Users\water\anaconda3\envs\DSE6211\Lib\site-packages\torch\nn\modules\module.py:1773: UserWarning: Implicit dimension choice for log_softmax has been deprecated. Change the call to include dim=X as an argument.

```
return self._call_impl(*args, **kwargs)
```

Epoch 14752 Training loss 0.0000 Test loss 0.0000 Train Slope: 0.0000000000000000
00 Test Slope: 0.00000000000000000023

Total training time: 16.912999153137207 seconds



Inference time for 3623 predictions: 0.7337698936462402 seconds

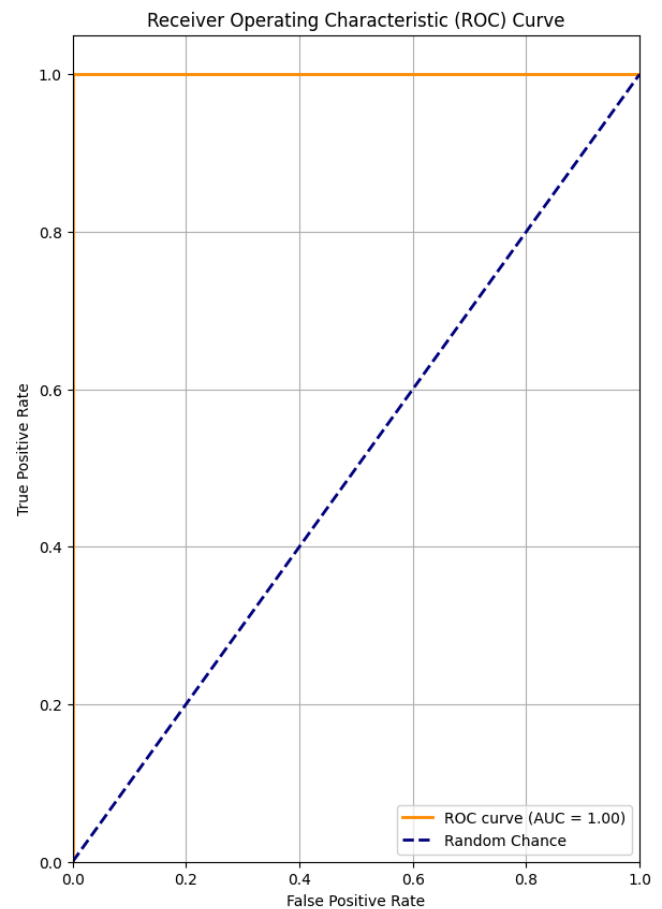
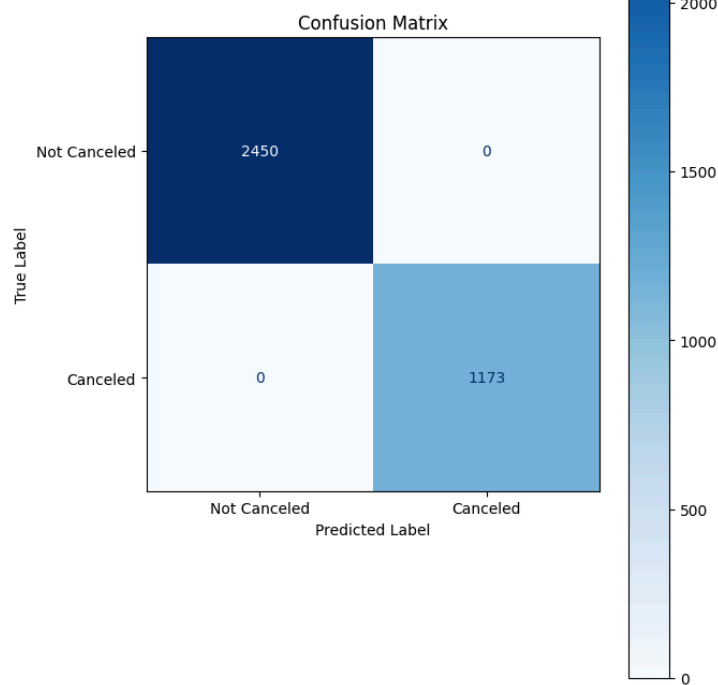
Number of Canceled Predictions: 1173
Number of Not Canceled Predictions: 2450
Percent Canceled Predictions: 32.37648357714601
Percent Not Canceled Predictions: 67.623516422854

Accuracy: 100.00%

Recall: 1.0

Precision: 1.0

F1-Score: 1.00



```
In [42]: model_4 = nn.Sequential(OrderedDict([
    ('hidden_linear1', nn.Linear(in_features=input_features, out_features=10)),
    ('hidden_linear2', nn.Linear(in_features=10, out_features=20)),
    ('hidden_linear3', nn.Linear(in_features=20, out_features=30)),
    ('hidden_linear4', nn.Linear(in_features=30, out_features=20)),
    ('hidden_linear5', nn.Linear(in_features=20, out_features=2)),
    ('output', nn.LogSoftmax())
]))

model_4 = ModelHandler(
    X = X,
    Y = Y,
    architecture = model_4,
    optimizer = optim.AdamW,
    loss_fn = nn.NLLLoss
)

model_4.train(1000000, 1e-4, 100, 1e-3)
model_4.eval_model(labels)
```

Train, Test, and Validation data is verified as unique.

Epoch 191 Training loss 0.3552 Test loss 0.3570

Train Slope: -0.00286655366420745

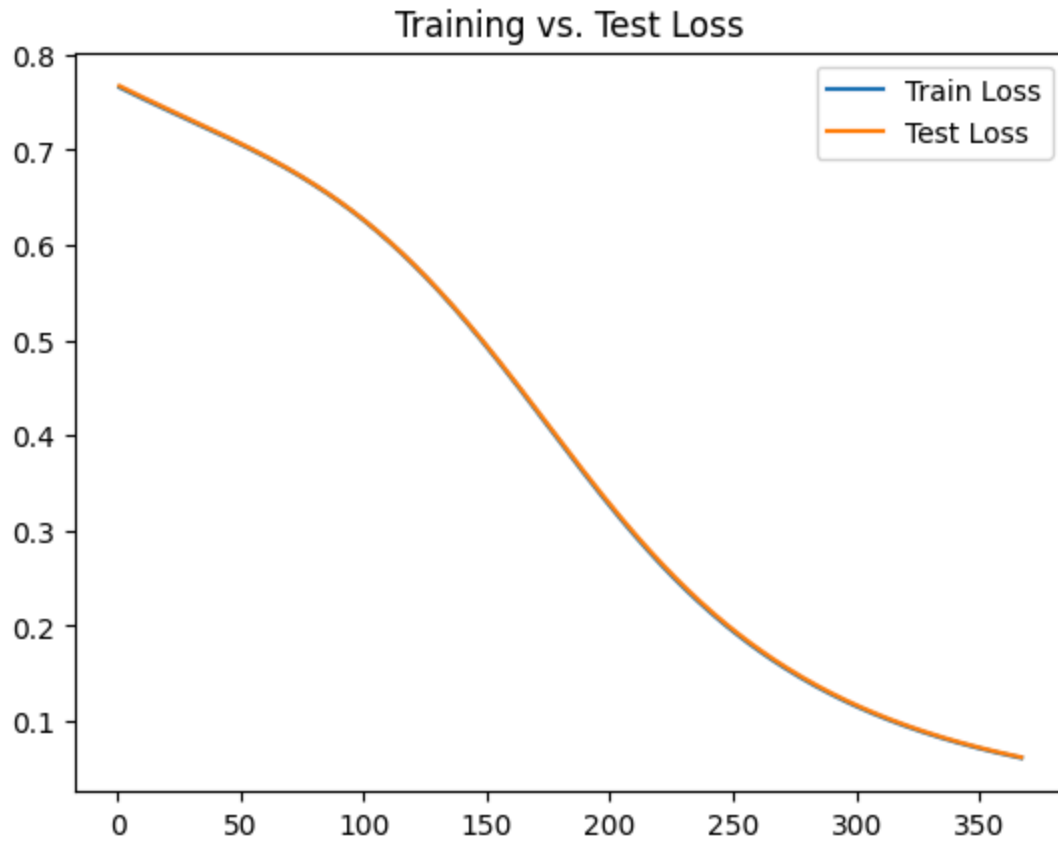
855 Test Slope: -0.00286655366420745855

c:\Users\water\anaconda3\envs\DSE6211\Lib\site-packages\torch\nn\modules\module.py:1773: UserWarning: Implicit dimension choice for log_softmax has been deprecated. Change the call to include dim=X as an argument.

return self._call_impl(*args, **kwargs)

Epoch 367 Training loss 0.0611 Test loss 0.0618 Train Slope: -0.00099455449730157
853 Test Slope: -0.00099455449730157853

Total training time: 0.4101448059082031 seconds



Inference time for 3623 predictions: 0.7331192493438721 seconds

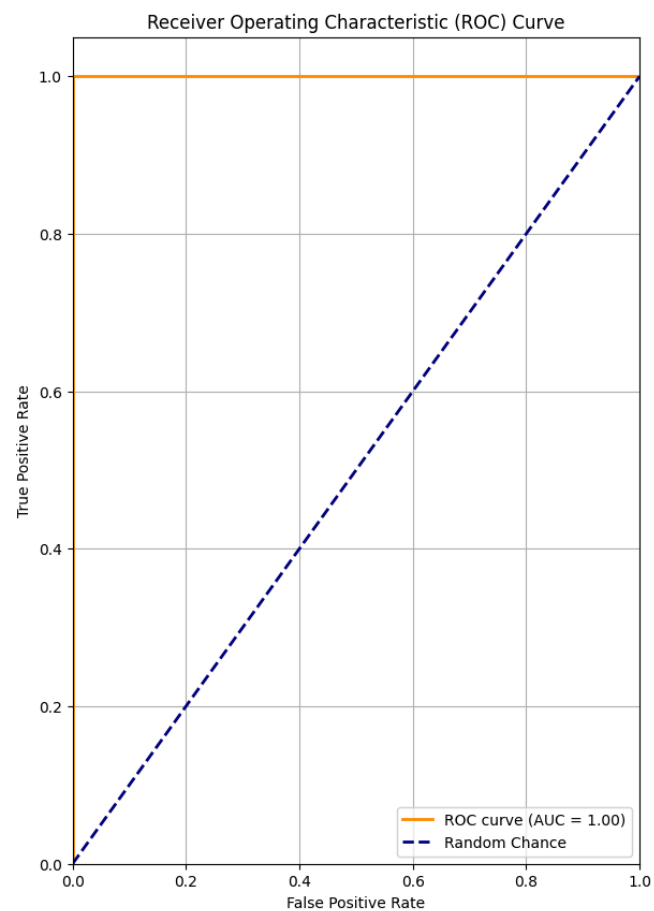
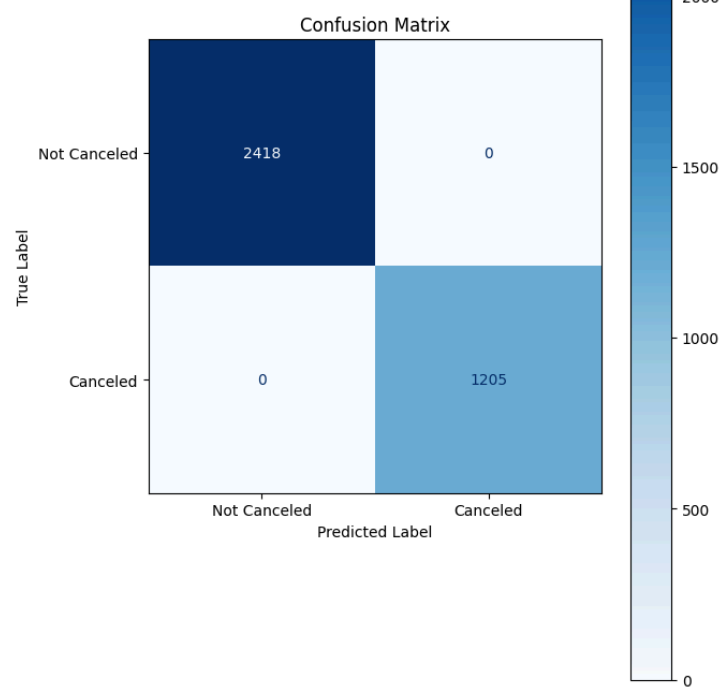
Number of Canceled Predictions: 1205
Number of Not Canceled Predictions: 2418
Percent Canceled Predictions: 33.25972950593431
Percent Not Canceled Predictions: 66.74027049406568

Accuracy: 100.00%

Recall: 1.0

Precision: 1.0

F1-Score: 1.00



In []: